

---

# When History Matters: Benchmarking Recurrent RL Agents on Memory Tasks

---

**Anh Khoa Pham**  
khoa.pham@stud.uni-hannover.de

**Quang Ky Anh Nguyen**  
ky.nguyen@stud.uni-hannover.de

## 1 Motivation

Standard RL algorithms like PPO, DQN assume the Markov property: the optimal action depends only on the current state. However, many environments are partially observable, means, the current observation alone is insufficient to act optimally. State aliasing occurs when different true states look identical to the agent, yet require different actions.

**Example:** In MiniGrid-Memory, the agent must remember an object it saw earlier, then select the matching object later. The choice point produces identical observations regardless of what was seen before, memoryless PPO and DQN can only guess to solve the task, find the same object as seen before.

We address this by introducing recurrent agents that maintain a hidden state encoding observation history. Instead of deciding actions from observation alone, they condition on both past hidden state and current observation.

**Research Question:** Does equipping PPO and DQN with recurrent memory recover performance in partially observable environments, and how much memory capacity and history length are actually needed?

We benchmark recurrent agents against baselines on memory-critical environments with ablations controlling observability level, memory capacity, and BPTT truncation horizon.

## 2 Related Topics

In the reinforcement learning course, algorithms such as DQN and PPO have been introduced and applied to simple environments like LunarLander and CartPole. These algorithms operate under the MDP assumption.

In Deep Learning, recurrent neural network layers such as LSTM and GRU possess the ability to maintain and integrate information from previous time steps, preserving temporal dependencies and event ordering. This property has been successfully applied in domains such as natural language processing.

The seminal paper "Deep Recurrent Q-Learning for Partially Observable MDPs" by Hausknecht et al. first combined DQN with LSTM in the Atari 2600 Game environment under the POMDP assumption [1]. More recently, "Generalization, Mayhem and Limits in Recurrent Proximal Policy Optimization" by Pleines et al. (2022) introduced RPPO, which integrates PPO with LSTM/GRU architectures for partially observable environments [2].

### 3 Experiments

#### 3.1 Environment

We adopt the Memory MiniGrid environment, where the agent begins in a left room with a  $7 \times 7$  observation field. The task is to identify an object (green key) in the initial room, then proceed to a fork on the right to select the matching object. This instantiates a POMDP: task completion depends on the agent remembering the object’s features across time steps. Reward is sparse, provided only upon task success. As shown in Fig. 1, we start with MiniGrid-MemoryS7-v0 and plan to scale to MiniGrid-MemoryS9-v0 or MiniGrid-MemoryS11-v0 if preliminary results are favorable.

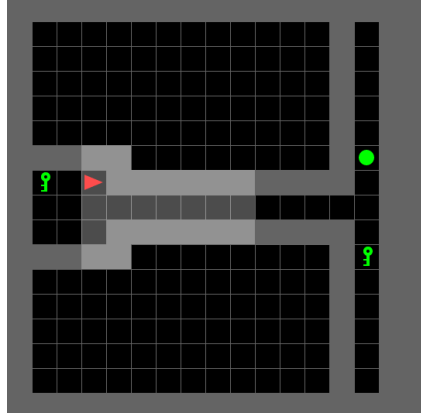


Figure 1: An example of Memory Minigrid environment

#### 3.2 RPPO

We employ a CNN1D encoder combined with an LSTM layer, shared between the actor and critic networks. Both networks use two separate heads for policy and value outputs, following the architecture proposed by Hausknecht et al.

**Sequence Padding:** Since episodes have variable lengths until termination, we pad all sequences to a uniform length to satisfy LSTM’s input requirements. Embeddings corresponding to padded positions are masked and excluded from loss computation across all three PPO losses (policy loss, value loss, and entropy bonus).

**Masking Example:** Given batch size 2:

$$\text{Episode 1: } [t_1, t_2, t_3, \text{PAD}, \text{PAD}] \quad (1)$$

$$\text{Episode 2: } [t_1, t_2, t_3, t_4, t_5] \quad (2)$$

After LSTM encoding and masking:

$$\text{Episode 1: } [0.8, 0.6, 0.4, 0.0, 0.0] \quad (3)$$

$$\text{Episode 2: } [0.5, 0.3, 0.7, 0.4, 0.6] \quad (4)$$

The effective batch contains 8 steps (3 from Episode 1, 5 from Episode 2). The remaining PPO loss terms remain unchanged.

#### 3.3 Hyperparameter Optimization

We conduct hyperparameter tuning for the LSTM architecture, focusing on input size, hidden size, and sequence length to analyze their impact on training dynamics. For PPO, we tune only the learning rate. The objective function is the total episode reward.

### 3.4 Benchmarking

We benchmark RPPO against PPO and DQN baselines under the POMDP assumption on Memory MiniGrid. We expect RPPO to substantially outperform memoryless PPO and DQN. All methods are subjected to hyperparameter optimization and evaluated across multiple random seeds.

## 4 Timeline

Our project timeline is as follows:

- Literature review: 3 days
- Implementation: 4 days
- Running code: 1 week
- Reporting, visualization, and poster design: 2 days

## References

- [1] M. J. Hausknecht and P. Stone, “Deep recurrent q-learning for partially observable mdps.” in *AAAI fall symposia*, vol. 45, 2015, p. 141.
- [2] M. Pleines, M. Pallasch, F. Zimmer, and M. Preuss, “Generalization, mayhems and limits in recurrent proximal policy optimization,” *arXiv preprint arXiv:2205.11104*, 2022.